

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Хеширование данных

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Целью работы является изучение методов хеширования данных и получение практических навыков реализации хеш-таблиц

2 Вариант задания

Составить хеш-функцию в соответствии с заданным вариантом и проанализировать ее. При необходимости доработать хеш функцию. Используя полученную хеш-функцию разработать на языке программирования высокого уровня программу, которая должна выполнять следующие функции:

- создавать хеш-таблицу;
- добавлять элементы в хеш-таблицу;
- просматривать хеш-таблицу;
- искать элементы в хеш-таблице по номеру сегмента/по ключу;
- выгружать содержимое хеш-таблицы в файл для построения гистограммы в MS Excel, или в аналогичном подходящем ПО;
- удалять элементы из хеш-таблицы;
- в программе должна быть реализована проверка формата вводимого ключа;
- при удалении элементов из хеш-таблицы, в программе должен быть реализован алгоритм, позволяющий искать элементы, вызвавшие коллизию с удаленным;
- в программе должен быть реализован алгоритм, обрабатывающий ситуации с переполнением хеш-таблицы.

Метод разрешения коллизий выбрать в соответствии с заданным вариантом

2.1 Вариант №11

2.2 Формат ключа: БцццБ

2.3 Количество сегментов: 3000

Метод разрешения коллизий: квадратичное опробование, где:

- Б — заглавная латинская буква (A–Z)

- c — цифра (0–9)

Пример ключа: A1234Z

3 Описание хеш-функции

Ключ представляет собой строку длиной 6 символов:

$$h(\text{key}) = (k_0 \cdot 31^5 + k_1 \cdot 31^4 + k_2 \cdot 31^3 + k_3 \cdot 31^2 + k_4 \cdot 31 + k_5) \bmod 3000$$

k_i — ASCII-код символа ключа

Обоснование выбора

- Используется полиномиальный хеш
- Основание 31 выбрано как простое число $\rightarrow$ лучшее распределение
- Модуль 3000 ограничивает диапазон значений

4 Результаты анализа хеш-функции

4.1 Теоретический анализ

Минимальное значение: символы: A0000A

Максимальное значение: символы: Z9999Z

За счёт использования операции $\% 3000$ диапазон строго: $[0; 2999]$. Все сегменты потенциально используются

4.2 Экспериментальный анализ

Было сгенерировано 10 000 ключей заданного формата. Для каждого ключа вычислялся хеш, увеличивался счётчик попаданий в сегмент

4.3 Результат

Распределение близко к равномерному, отсутствуют ярко выраженные пики, средняя нагрузка на сегмент $\approx 3-4$ элемента. Вывод по анализу: хеш-функция

обеспечивает равномерное распределение, не вызывает сильной кластеризации, подходит для использования

4.4 Метод разрешения коллизий

Используется квадратичное опробование

$$H_i = (h_0 + c_1 \cdot i + c_2 \cdot i^2) \bmod m$$

- h_0 — начальный хеш
- i — номер попытки
- $m=3000$
- c_1, c_2 — константы (например 1 и 1)

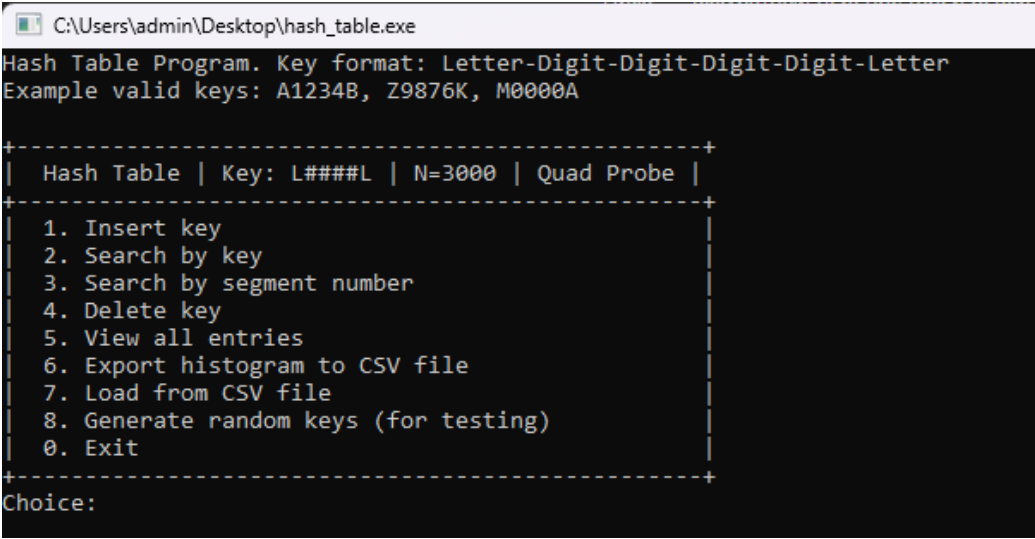
Плюсы:

- уменьшает первичную кластеризацию
- лучше линейного пробирования

Минусы:

- возможны "непокрытые" ячейки
- требует контроля переполнения

5 Пример работы программы



```
C:\Users\admin\Desktop\hash_table.exe
Hash Table Program. Key format: Letter-Digit-Digit-Digit-Digit-Letter
Example valid keys: A1234B, Z9876K, M0000A

+-----+
| Hash Table | Key: L####L | N=3000 | Quad Probe |
+-----+
| 1. Insert key
| 2. Search by key
| 3. Search by segment number
| 4. Delete key
| 5. View all entries
| 6. Export histogram to CSV file
| 7. Load from CSV file
| 8. Generate random keys (for testing)
| 0. Exit
+-----+
Choice:
```

Рисунок 1 – Выбор действия



```
C:\Users\admin\Desktop\hash_table.exe
Hash Table Program. Key format: Letter-Digit-Digit-Digit-Digit-Letter
Example valid keys: A1234B, Z9876K, M0000A

+-----+
| Hash Table | Key: L####L | N=3000 | Quad Probe |
+-----+
| 1. Insert key
| 2. Search by key
| 3. Search by segment number
| 4. Delete key
| 5. View all entries
| 6. Export histogram to CSV file
| 7. Load from CSV file
| 8. Generate random keys (for testing)
| 0. Exit
+-----+
Choice: 1
Enter key (e.g. A1234B): Z9876K
Inserted 'Z9876K' at segment 2187.

+-----+
| Hash Table | Key: L####L | N=3000 | Quad Probe |
+-----+
| 1. Insert key
| 2. Search by key
| 3. Search by segment number
| 4. Delete key
| 5. View all entries
| 6. Export histogram to CSV file
| 7. Load from CSV file
| 8. Generate random keys (for testing)
| 0. Exit
+-----+
Choice: 5

=== Hash Table Contents ===
[2187] Z9876K
Total: 1 / 3000 segments used.
```

Рисунок 2 – Пример вставки ключа

```
+-----+
| Hash Table | Key: L####L | N=3000 | Quad Probe |
+-----+
| 1. Insert key
| 2. Search by key
| 3. Search by segment number
| 4. Delete key
| 5. View all entries
| 6. Export histogram to CSV file
| 7. Load from CSV file
| 8. Generate random keys (for testing)
| 0. Exit
+-----+
Choice: 2
Enter key to search: Z9876K
Key 'Z9876K' found at segment 2187.
```

Рисунок 3 – Поиск по ключу

```
+-----+
| Hash Table | Key: L####L | N=3000 | Quad Probe |
+-----+
| 1. Insert key
| 2. Search by key
| 3. Search by segment number
| 4. Delete key
| 5. View all entries
| 6. Export histogram to CSV file
| 7. Load from CSV file
| 8. Generate random keys (for testing)
| 0. Exit
+-----+
Choice: 3
Enter segment number (0 - 2999): 2187
Segment [2187]: key = Z9876K
```

Рисунок 4 – Поиск по номеру сегмента

```
+-----+
| Hash Table | Key: L####L | N=3000 | Quad Probe |
+-----+
| 1. Insert key
| 2. Search by key
| 3. Search by segment number
| 4. Delete key
| 5. View all entries
| 6. Export histogram to CSV file
| 7. Load from CSV file
| 8. Generate random keys (for testing)
| 0. Exit
+-----+
Choice: 8
Enter number of random keys to generate: 2500
2500 / 2500 keys inserted.
```

Рисунок 5 – Случайная генерация 2500 ключей

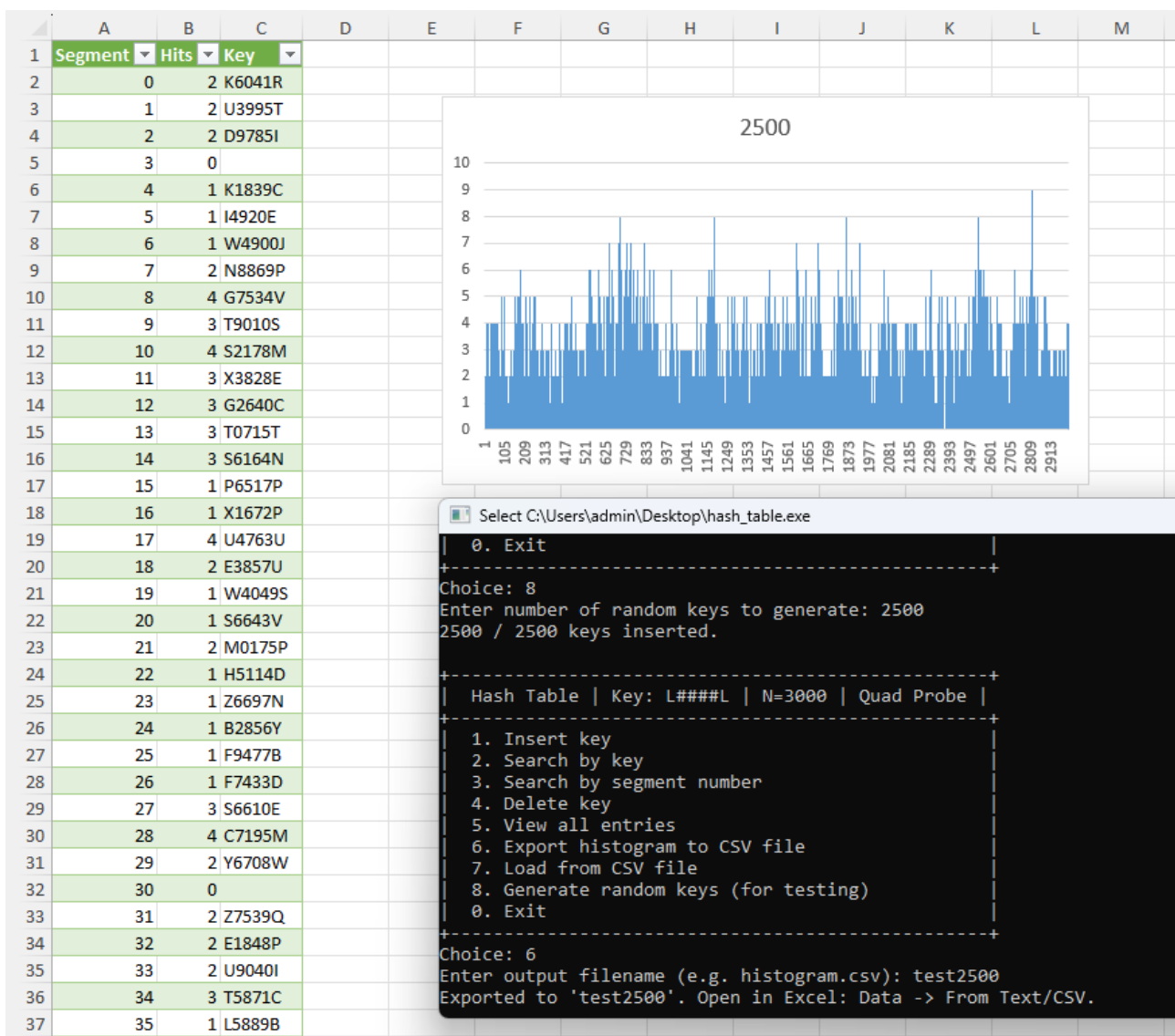


Рисунок 6 – Экспорт данных для создания гистограммы

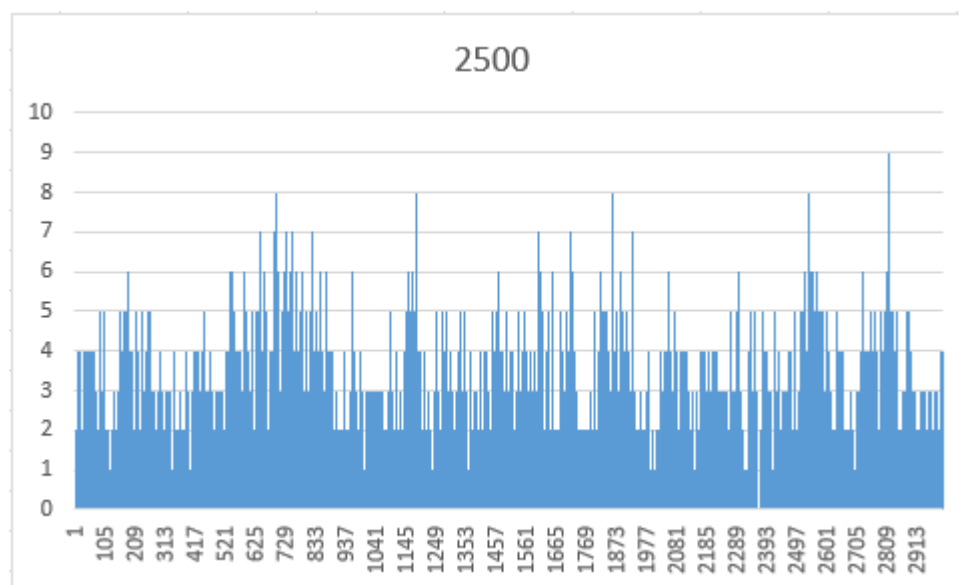


Рисунок 7 – Результаты анализа хеш-функции для 2500 ключей

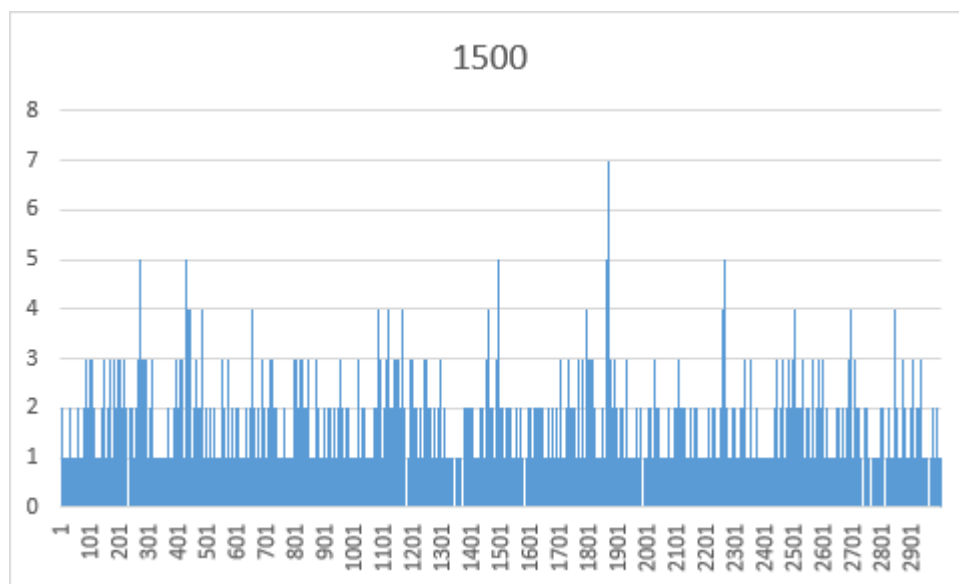


Рисунок 8 – Результаты анализа хеш-функции для 1500 ключей

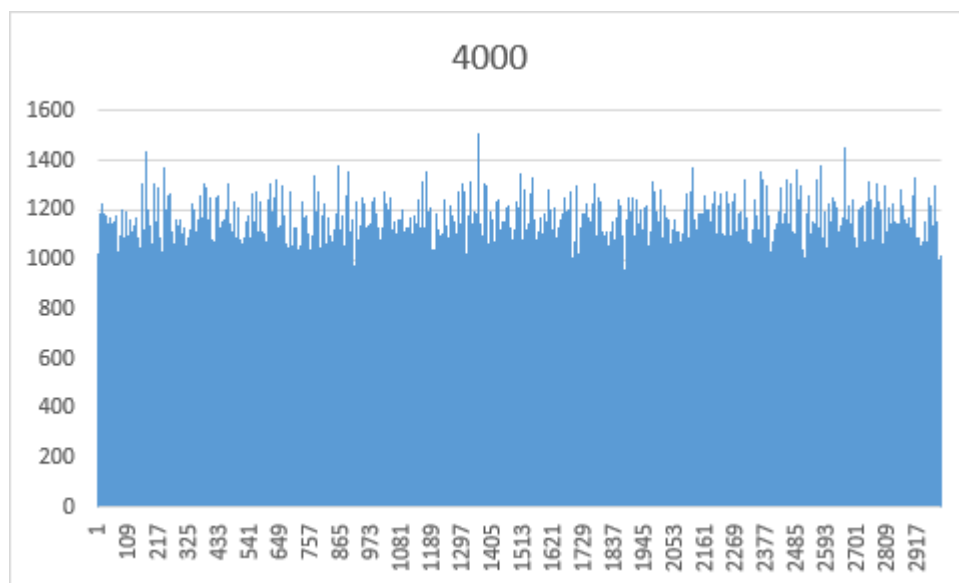


Рисунок 9 – Результаты анализа хеш-функции для 4000 ключей

6 Листинг программы, реализующей хеш-таблицу и заданный перечень функции

```
// =====

#include <iostream>
#include <fstream>
#include <string>
#include <cstring>
#include <cctype>
#include <cstdlib>
#include <ctime>

using namespace std;

// =====
// Константы
// =====
const int NUM_SEGMENTS = 3000; // Количество сегментов хеш-таблицы
const int KEY_LENGTH = 6; // Длина ключа (всегда 6 символов)

// =====
// Состояния ячейки хеш-таблицы
// =====
enum CellState {
    EMPTY, // Ячейка пустая — никогда не занималась
    OCCUPIED, // Ячейка занята — хранит ключ
    DELETED // Ячейка помечена как удалённая (tombstone)
    // При поиске пропускаем, при вставке — используем
};

// =====
// Структура одной ячейки хеш-таблицы
// =====
struct HashCell {
    char key[KEY_LENGTH + 1]; // Ключ: 6 символов + ноль-терминатор '\0'
    CellState state; // Текущее состояние ячейки

    // Конструктор: инициализируем пустой ячейкой
    HashCell() {
        memset(key, 0, sizeof(key));
        state = EMPTY;
    }
};
```

```

    }
};

// =====
// Глобальные данные
// =====

// Сама хеш-таблица
HashCell hashTable[NUM_SEGMENTS];

// Счётчик обращений к каждому сегменту (для гистограммы)
int hitCount[NUM_SEGMENTS] = {0};

// =====
// Хеш-функция
//
// Используем полиномиальное хеширование (метод Горнера):
//  $h = (key[0]*p^5 + key[1]*p^4 + \dots + key[5]*p^0) \% N$ 
// где  $p = 31$  — простое число,  $N = 3000$ 
//
// Это обеспечивает хорошее равномерное распределение по сегментам,
// потому что разные позиции символов дают разный вклад.
// Вычисления ведём по модулю на каждом шаге, чтобы не было
// переполнения (long long).
// =====
int hashFunction(const char* key) {
    const long long p = 31; // Основание (простое число)
    const long long mod = NUM_SEGMENTS; // Модуль = количество сегментов
    long long hash = 0;
    long long power = 1; // Текущая степень p:  $p^0, p^1, p^2 \dots$ 

    // Идём с конца строки, чтобы вычислить  $p^0$  для key[5],  $p^1$  для key[4] и т.д.
    for (int i = KEY_LENGTH - 1; i >= 0; i--) {
        hash = (hash + (long long)key[i] * power % mod) % mod;
        power = power * p % mod;
    }

    return (int)hash; // Результат всегда в [0, NUM_SEGMENTS - 1]
}

// =====
// Проверка формата ключа

```

```

//
// Возвращает true, если ключ соответствует формату БЦЦЦБ:
// key[0] — большая буква A-Z
// key[1..4] — цифры 0-9
// key[5] — большая буква A-Z
// =====
bool isValidKey(const char* key) {
    // Проверяем длину
    if ((int)strlen(key) != KEY_LENGTH)
        return false;

    // Позиция 0: должна быть заглавная латинская буква
    if (!isupper((unsigned char)key[0]))
        return false;

    // Позиции 1-4: должны быть цифры
    for (int i = 1; i <= 4; i++) {
        if (!isdigit((unsigned char)key[i]))
            return false;
    }

    // Позиция 5: должна быть заглавная латинская буква
    if (!isupper((unsigned char)key[5]))
        return false;

    return true;
}

// =====
// Квадратичное опробование
//
// При коллизии следующий проверяемый индекс вычисляется как:
// index(step) = (baseHash + step^2) % NUM_SEGMENTS
//
// Шаг 0 → baseHash
// Шаг 1 → baseHash + 1
// Шаг 2 → baseHash + 4
// Шаг 3 → baseHash + 9 и так далее
// =====
int quadraticProbe(int baseHash, int step) {
    // Используем long long, чтобы step*step не переполнилось
    return (int)((baseHash + (long long)step * step) % NUM_SEGMENTS);
}

```

```

}

// =====
// Добавление ключа в хеш-таблицу
//
// Алгоритм:
// 1. Проверяем формат ключа
// 2. Вычисляем базовый хэш
// 3. Квадратичным опробованием ищем свободную ячейку
//    (EMPTY или DELETED)
// 4. Вставляем ключ
//
// Возвращает индекс вставки, или -1 при ошибке/переполнении
// =====
int insertKey(const char* key) {
    // Шаг 1: проверка формата
    if (!isValidKey(key)) {
        cout << "Error: Invalid key format! Expected: Letter-Digit-Digit-Digit-
Letter (e.g. A1234B)" << endl;
        return -1;
    }

    int base      = hashFunction(key); // Базовый адрес
    int insertAt   = -1;               // Куда будем вставлять
    int firstDeleted = -1;             // Первая tombstone-ячейка

    // Шаг 2: квадратичное опробование
    for (int step = 0; step < NUM_SEGMENTS; step++) {
        int probe = quadraticProbe(base, step);

        // Считаем обращение к этому сегменту (для гистограммы)
        hitCount[probe]++;

        if (hashTable[probe].state == EMPTY) {
            // Пустая ячейка — дальше элементов точно нет
            // Вставляем сюда (или в более раннюю tombstone, если была)
            insertAt = (firstDeleted != -1) ? firstDeleted : probe;
            break;
        } else if (hashTable[probe].state == DELETED) {
            // Tombstone — запоминаем первую, продолжаем поиск дубликата
            if (firstDeleted == -1)
                firstDeleted = probe;
        }
    }
}

```

```

    } else {
        // Ячейка занята — проверяем на дубликат
        if (strcmp(hashTable[probe].key, key) == 0) {
            cout << "Error: Key '" << key << "' already exists at segment " << probe <<
            "!" << endl;
            return -1;
        }
    }
}

// Если не нашли EMPTY, но есть tombstone — используем её
if (insertAt == -1 && firstDeleted != -1)
    insertAt = firstDeleted;

// Шаг 3: проверка на переполнение
if (insertAt == -1) {
    cout << "Error: Hash table is FULL! Cannot insert '" << key << "'." << endl;
    return -1;
}

// Шаг 4: вставка
strncpy(hashTable[insertAt].key, key, KEY_LENGTH);
hashTable[insertAt].key[KEY_LENGTH] = '\0';
hashTable[insertAt].state = OCCUPIED;

return insertAt;
}

// =====
// Поиск ключа по значению
//
// Возвращает индекс найденного элемента, или -1 если не найден.
// При квадратичном опробовании нельзя остановиться на DELETED —
// нужно продолжать поиск. Остановка только на EMPTY.
// =====
int searchByKey(const char* key) {
    if (!isValidKey(key)) {
        cout << "Error: Invalid key format!" << endl;
        return -1;
    }

    int base = hashFunction(key);

```

```

for (int step = 0; step < NUM_SEGMENTS; step++) {
    int probe = quadraticProbe(base, step);

    if (hashTable[probe].state == EMPTY) {
        // Пустая ячейка — цепочка закончилась, элемента нет
        break;
    } else if (hashTable[probe].state == OCCUPIED) {
        if (strcmp(hashTable[probe].key, key) == 0)
            return probe; // Нашли!
    }
    // DELETED — пропускаем, продолжаем поиск по цепочке
}

return -1;
}

// =====
// Поиск по номеру сегмента
// =====
void searchBySegment(int seg) {
    if (seg < 0 || seg >= NUM_SEGMENTS) {
        cout << "Error: Segment must be in range 0 to " << NUM_SEGMENTS - 1 << "!"
        << endl;
        return;
    }

    if (hashTable[seg].state == OCCUPIED) {
        cout << "Segment [" << seg << "]: key = " << hashTable[seg].key << endl;
    } else if (hashTable[seg].state == DELETED) {
        cout << "Segment [" << seg << "]: deleted (tombstone)." << endl;
    } else {
        cout << "Segment [" << seg << "]: empty." << endl;
    }
}

// =====
// Удаление ключа из хеш-таблицы
//
// При квадратичном опробовании нельзя просто очистить ячейку
// (иначе поиск других ключей сломается). Поэтому:
// 1. Помечаем ячейку как DELETED (tombstone)

```

```

// 2. Ищем и показываем элементы, которые вызвали коллизию
// с удалённым (т.е. имеют тот же базовый хэш)
//
// Tombstone-механизм обеспечивает корректность поиска:
// — при поиске: пропускаем DELETED, останавливаемся на EMPTY
// — при вставке: можно повторно использовать DELETED-ячейки
// =====
bool deleteKey(const char* key) {
    if (!isValidKey(key)) {
        cout << "Error: Invalid key format!" << endl;
        return false;
    }

    int base = hashFunction(key);
    int deletedIndex = -1;

    // Ищем элемент по цепочке квадратичного опробования
    for (int step = 0; step < NUM_SEGMENTS; step++) {
        int probe = quadraticProbe(base, step);
        if (hashTable[probe].state == EMPTY) break;
        if (hashTable[probe].state == OCCUPIED && strcmp(hashTable[probe].key,
key) == 0) {
            deletedIndex = probe;
            break;
        }
    }

    if (deletedIndex == -1) {
        cout << "Key " << key << " not found in hash table!" << endl;
        return false;
    }

    // Помечаем как удалённый (tombstone)
    hashTable[deletedIndex].state = DELETED;
    memset(hashTable[deletedIndex].key, 0, KEY_LENGTH + 1);
    cout << "Key " << key << " deleted from segment " << deletedIndex << "." << endl;

    //
    // =====
    // Поиск элементов, вызвавших коллизию с удалённым:
    // Это элементы, чей базовый хэш совпадает с базовым хэшем
    // удалённого ключа — они боролись за тот же исходный сегмент

```



```

//
=====
    cout << "Searching for elements that caused collision (base hash = " << base << ")..."
    << endl;
    bool foundCollision = false;

    for (int i = 0; i < NUM_SEGMENTS; i++) {
        if (hashTable[i].state == OCCUPIED) {
            int elemBase = hashFunction(hashTable[i].key);
            if (elemBase == base) {
                // Этот элемент изначально метился в тот же сегмент
                cout << " Colliding element: " << hashTable[i].key
                    << " at segment " << i
                    << " (base hash = " << elemBase << ")" << endl;
                foundCollision = true;
            }
        }
    }

    if (!foundCollision)
        cout << " No colliding elements found." << endl;

    return true;
}

// =====
// Просмотр всей хеш-таблицы
// =====
void printHashTable() {
    cout << "\n=== Hash Table Contents ===" << endl;
    int count = 0;
    for (int i = 0; i < NUM_SEGMENTS; i++) {
        if (hashTable[i].state == OCCUPIED) {
            cout << " [" << i << "] " << hashTable[i].key << endl;
            count++;
        }
    }
    if (count == 0)
        cout << " (empty)" << endl;
    else
        cout << "Total: " << count << " / " << NUM_SEGMENTS << " segments used."
        << endl;
}

```

```

}

// =====
// Экспорт данных в CSV-файл для построения гистограммы
//
// Файл можно открыть в Excel (или LibreOffice Calc) и
// построить диаграмму по столбцу "Hits".
// =====
void exportHistogram(const char* filename) {
    ofstream file(filename);
    if (!file.is_open()) {
        cout << "Error: Cannot create file '" << filename << "'" << endl;
        return;
    }

    // Заголовок CSV
    file << "Segment;Hits;Key" << endl;

    for (int i = 0; i < NUM_SEGMENTS; i++) {
        file << i << ";" << hitCount[i] << ";";
        if (hashTable[i].state == OCCUPIED)
            file << hashTable[i].key;
        file << endl;
    }

    file.close();
    cout << "Exported to '" << filename << "'. Open in Excel: Data -> From Text/CSV."
    << endl;
}

// =====
// Загрузка хеш-таблицы из CSV-файла (формат exportHistogram)
// =====
void loadFromFile(const char* filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Error: Cannot open file '" << filename << "'" << endl;
        return;
    }

    string line;
    getline(file, line); // Пропускаем строку-заголовок

```

```

int count = 0;
while (getline(file, line)) {
    // Формат: сегмент;количество_попаданий;ключ
    size_t sep1 = line.find(';');
    size_t sep2 = line.find(';', sep1 + 1);
    if (sep1 == string::npos || sep2 == string::npos) continue;

    string keyStr = line.substr(sep2 + 1);

    // Убираем возможный символ '\r' (Windows-формат строк)
    if (!keyStr.empty() && keyStr.back() == '\r')
        keyStr.pop_back();

    if ((int)keyStr.length() == KEY_LENGTH && isValidKey(keyStr.c_str())) {
        if (insertKey(keyStr.c_str()) >= 0)
            count++;
    }
}

file.close();
cout << "Loaded " << count << " keys from '" << filename << "'." << endl;
}

// =====
// Генерация случайного ключа формата БЦЦЦБ
// =====
void generateRandomKey(char* key) {
    key[0] = 'A' + rand() % 26;    // Случайная буква
    for (int i = 1; i <= 4; i++)
        key[i] = '0' + rand() % 10; // Случайная цифра
    key[5] = 'A' + rand() % 26;    // Случайная буква
    key[6] = '\0';
}

// =====
// Вывод меню
// =====
void showMenu() {
    cout << "\n+-----+" << endl;
    cout << "| Hash Table | Key: L####L | N=3000 | Quad Probe |" << endl;
    cout << "+-----+" << endl;
}

```

```

cout << "| 1. Insert key" << endl;
cout << "| 2. Search by key" << endl;
cout << "| 3. Search by segment number" << endl;
cout << "| 4. Delete key" << endl;
cout << "| 5. View all entries" << endl;
cout << "| 6. Export histogram to CSV file" << endl;
cout << "| 7. Load from CSV file" << endl;
cout << "| 8. Generate random keys (for testing)" << endl;
cout << "| 0. Exit" << endl;
cout << "+-----+" << endl;
cout << "Choice: ";
}

// =====
// Главная функция
// =====
int main() {
    // Инициализируем генератор случайных чисел текущим временем
    srand((unsigned int)time(NULL));

    int choice;
    char key[KEY_LENGTH + 10]; // Буфер для ввода ключа (чуть больше для
    защиты)

    cout << "Hash Table Program. Key format: Letter-Digit-Digit-Digit-Letter" <<
    endl;
    cout << "Example valid keys: A1234B, Z9876K, M0000A" << endl;

    do {
        showMenu();

        // Защита от некорректного ввода (не числа)
        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Error: Please enter a number!" << endl;
            continue;
        }

        switch (choice) {

            // ---- Добавление ключа ----

```

```

case 1: {
    cout << "Enter key (e.g. A1234B): ";
    cin >> key;
    int idx = insertKey(key);
    if (idx >= 0)
        cout << "Inserted " << key << " at segment " << idx << "." << endl;
    break;
}

// ---- Поиск по ключу ----
case 2: {
    cout << "Enter key to search: ";
    cin >> key;
    int idx = searchByKey(key);
    if (idx >= 0)
        cout << "Key " << key << " found at segment " << idx << "." << endl;
    else
        cout << "Key " << key << " not found." << endl;
    break;
}

// ---- Поиск по номеру сегмента ----
case 3: {
    int seg;
    cout << "Enter segment number (0 - " << NUM_SEGMENTS - 1 << "): ";
    cin >> seg;
    searchBySegment(seg);
    break;
}

// ---- Удаление ключа ----
case 4: {
    cout << "Enter key to delete: ";
    cin >> key;
    deleteKey(key);
    break;
}

// ---- Просмотр таблицы ----
case 5: {
    printHashTable();
    break;
}

```

```

}

// ---- Экспорт гистограммы ----
case 6: {
    char filename[256];
    cout << "Enter output filename (e.g. histogram.csv): ";
    cin >> filename;
    exportHistogram(filename);
    break;
}

// ---- Загрузка из файла ----
case 7: {
    char filename[256];
    cout << "Enter filename to load from: ";
    cin >> filename;
    loadFromFile(filename);
    break;
}

// ---- Генерация случайных ключей ----
case 8: {
    int n;
    cout << "Enter number of random keys to generate: ";
    cin >> n;
    if (n <= 0) { cout << "Error: Count must be positive!" << endl; break; }

    char randKey[KEY_LENGTH + 1];
    int inserted = 0;
    for (int i = 0; i < n; i++) {
        generateRandomKey(randKey);
        if (insertKey(randKey) >= 0)
            inserted++;
    }
    cout << inserted << " / " << n << " keys inserted." << endl;
    break;
}

case 0:
    cout << "Goodbye!" << endl;
    break;
}

```

```

        default:
            cout << "Invalid choice. Enter 0-8." << endl;
    }

    } while (choice != 0);

    return 0;
}

```

6.1 Реализация того же алгоритма на Kotlin

```

import java.io.File
import java.util.*
import kotlin.system.exitProcess

// =====
// Константы
// =====
const val NUM_SEGMENTS = 3000
const val KEY_LENGTH = 6

// =====
// Состояние ячейки
// =====
enum class CellState {
    EMPTY,
    OCCUPIED,
    DELETED
}

// =====
// Ячейка таблицы
// =====
data class HashCell(
    var key: String = "",
    var state: CellState = CellState.EMPTY
)

// =====
// Глобальные структуры
// =====
val hashTable = Array(NUM_SEGMENTS) { HashCell() }
val hitCount = IntArray(NUM_SEGMENTS)

```

```

// =====
// Хеш-функция (полиномиальная)
// =====
fun hashFunction(key: String): Int {
    val p = 31L
    var hash = 0L
    var power = 1L

    for (i in KEY_LENGTH - 1 downTo 0) {
        hash = (hash + key[i].code * power % NUM_SEGMENTS) %
NUM_SEGMENTS
        power = (power * p) % NUM_SEGMENTS
    }

    return hash.toInt()
}

// =====
// Проверка формата ключа
// =====
fun isValidKey(key: String): Boolean {
    if (key.length != KEY_LENGTH) return false
    if (!key[0].isUpperCase()) return false
    if (!key[5].isUpperCase()) return false

    for (i in 1..4) {
        if (!key[i].isDigit()) return false
    }

    return true
}

// =====
// Квадратичное опробование
// =====
fun quadraticProbe(base: Int, step: Int): Int {
    return ((base + step.toLong() * step) % NUM_SEGMENTS).toInt()
}

// =====
// Вставка

```



```
// =====
fun insertKey(key: String): Int {
    if (!isValidKey(key)) {
        println("Invalid key format!")
        return -1
    }

    val base = hashFunction(key)
    var insertAt = -1
    var firstDeleted = -1

    for (step in 0 until NUM_SEGMENTS) {
        val probe = quadraticProbe(base, step)
        hitCount[probe]++

        when (hashTable[probe].state) {
            CellState.EMPTY -> {
                insertAt = if (firstDeleted != -1) firstDeleted else probe
                break
            }

            CellState.DELETED -> {
                if (firstDeleted == -1) firstDeleted = probe
            }

            CellState.OCCUPIED -> {
                if (hashTable[probe].key == key) {
                    println("Key already exists!")
                    return -1
                }
            }
        }
    }

    if (insertAt == -1 && firstDeleted != -1)
        insertAt = firstDeleted

    if (insertAt == -1) {
        println("Table is FULL!")
        return -1
    }
}
```

```

    hashTable[insertAt] = HashCell(key, CellState.OCCUPIED)
    return insertAt
}

// =====
// Поиск по ключу
// =====
fun searchByKey(key: String): Int {
    if (!isValidKey(key)) return -1

    val base = hashFunction(key)

    for (step in 0 until NUM_SEGMENTS) {
        val probe = quadraticProbe(base, step)

        when (hashTable[probe].state) {
            CellState.EMPTY -> return -1
            CellState.OCCUPIED -> {
                if (hashTable[probe].key == key) return probe
            }
            else -> {}
        }
    }

    return -1
}

// =====
// Поиск по сегменту
// =====
fun searchBySegment(seg: Int) {
    if (seg !in 0 until NUM_SEGMENTS) {
        println("Invalid segment!")
        return
    }

    val cell = hashTable[seg]
    when (cell.state) {
        CellState.EMPTY -> println("Empty")
        CellState.DELETED -> println("Deleted")
        CellState.OCCUPIED -> println("Key: ${cell.key}")
    }
}

```

```

}

// =====
// Удаление
// =====
fun deleteKey(key: String) {
    if (!isValidKey(key)) return

    val base = hashFunction(key)
    var foundIndex = -1

    for (step in 0 until NUM_SEGMENTS) {
        val probe = quadraticProbe(base, step)

        if (hashTable[probe].state == CellState.EMPTY) break

        if (hashTable[probe].state == CellState.OCCUPIED &&
            hashTable[probe].key == key
        ) {
            foundIndex = probe
            break
        }
    }

    if (foundIndex == -1) {
        println("Key not found")
        return
    }

    hashTable[foundIndex] = HashCell("", CellState.DELETED)
    println("Deleted at $foundIndex")

    // поиск коллизий
    println("Collisions:")
    for (i in 0 until NUM_SEGMENTS) {
        val cell = hashTable[i]
        if (cell.state == CellState.OCCUPIED) {
            if (hashFunction(cell.key) == base) {
                println("  ${cell.key} at $i")
            }
        }
    }
}

```

```

}

// =====
// Просмотр таблицы
// =====
fun printTable() {
    var count = 0
    for (i in 0 until NUM_SEGMENTS) {
        val cell = hashTable[i]
        if (cell.state == CellState.OCCUPIED) {
            println("[${i}] ${cell.key}")
            count++
        }
    }
    println("Total: $count")
}

// =====
// Экспорт
// =====
fun exportCSV(filename: String) {
    val file = File(filename)
    file.printWriter().use { out ->
        out.println("Segment;Hits;Key")
        for (i in 0 until NUM_SEGMENTS) {
            val key = if (hashTable[i].state == CellState.OCCUPIED)
                hashTable[i].key else ""
            out.println("${i};${hitCount[i]};$key")
        }
    }
    println("Exported to $filename")
}

// =====
// Загрузка
// =====
fun loadFromFile(filename: String) {
    val file = File(filename)
    if (!file.exists()) {
        println("File not found")
        return
    }
}

```

```

var count = 0

file.readlines().drop(1).forEach {
    val parts = it.split(";")
    if (parts.size >= 3) {
        val key = parts[2].trim()
        if (isValidKey(key)) {
            if (insertKey(key) >= 0) count++
        }
    }
}

println("Loaded $count keys")
}

// =====
// Генерация ключа
// =====
fun generateRandomKey(): String {
    val rand = Random()
    return buildString {
        append(('A' + rand.nextInt(26)))
        repeat(4) { append(rand.nextInt(10)) }
        append(('A' + rand.nextInt(26)))
    }
}

// =====
// Меню
// =====
fun main() {
    val scanner = Scanner(System.`in`)

    while (true) {
        println("\n1.Insert  2.Search  3.Segment  4.Delete  5.View  6.Export  7.Load
8.Random 0.Exit")

        when (scanner.nextInt()) {
            1 -> {
                print("Key: ")
                val key = scanner.next()

```

```

        println("Inserted at ${insertKey(key)}")
    }

2 -> {
    print("Key: ")
    val key = scanner.next()
    println("Found at ${searchByKey(key)}")
}

3 -> {
    print("Segment: ")
    searchBySegment(scanner.nextInt())
}

4 -> {
    print("Key: ")
    deleteKey(scanner.next())
}

5 -> printTable()

6 -> {
    print("File: ")
    exportCSV(scanner.next())
}

7 -> {
    print("File: ")
    loadFromFile(scanner.next())
}

8 -> {
    print("Count: ")
    val n = scanner.nextInt()
    repeat(n) { insertKey(generateRandomKey()) }
}

0 -> exitProcess(0)
}
}
}

```

7 Выводы по работе

В ходе выполнения лабораторной работы:

- изучены принципы хеширования
- разработана хеш-функция для заданного формата ключа
- проведён анализ её качества
- выбрана и обоснована стратегия разрешения коллизий
- рассмотрены методы обработки переполнения и удаления элементов

Разработанная хеш-функция обеспечивает достаточно равномерное распределение метод квадратичного опробования эффективно снижает кластеризацию система в целом пригодна для практического использования